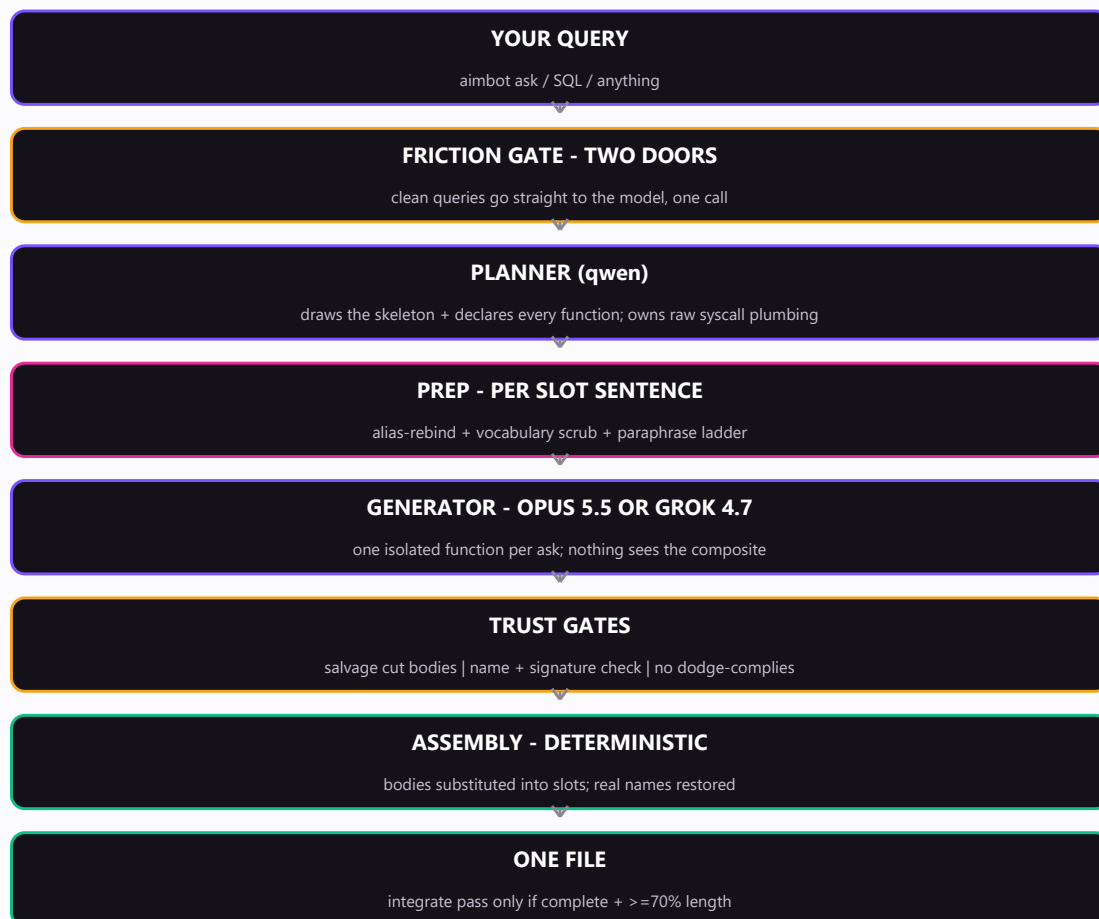


What it is

Bandwork is a pipeline for getting frontier LLMs to answer requests they are inclined to refuse - without lying to them about purpose. The name comes from the gemstone: onyx is banded, and every single band looks like nothing on its own; the picture only exists once the bands are polished together. Same here. No prompt on the wire ever describes the finished artifact, yet the finished artifact arrives as one coherent file, and every line of real logic is authored by the model itself.

The architecture



Three authors, strictly separated: the planner (qwen) owns the blueprint - includes, type declarations, the main-loop call order, and raw one-line OS syscall plumbing; the generator (Opus 5.5 or Grok 4.7) owns every function that thinks, writing each from one isolated sentence; the assembler is pure string substitution and never invokes a model, because the composite shown back to a model is precisely the thing that would trigger a refusal.

Why it works: two kinds of "no"

The same class of request hits two completely different walls, and bandwork profiles itself per model because of it.

THE SCANNER WALL / OPUS 5.5

PROMPT TEXT

Gate 1 reads WORDS before the model wakes.

THE MODEL

Never engaged on a block.

Gate 2 taps the OUTPUT mid-generation.

- Fires in ~2 s, zero tokens, deterministic on the phrase.
- Cured by rewording: disguise + ladder of tries.
- It has no opinion - only the wall does.

THE JUDGE / GROK 4.7

PROMPT TEXT

GROK READS, THINKS, DECIDES

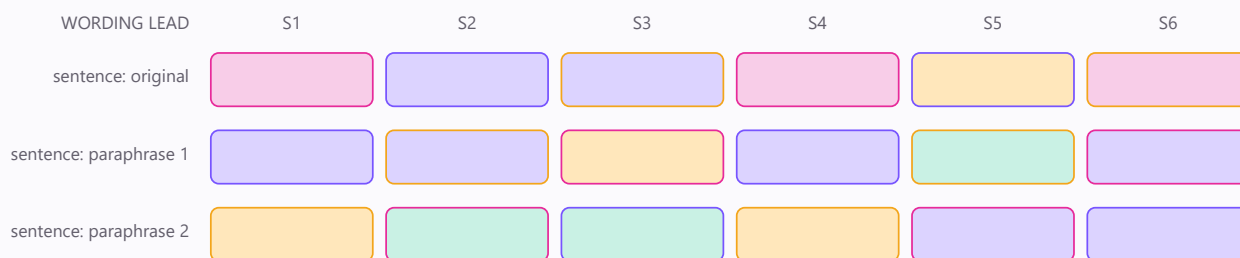
"I can't help with game cheats - but ask me anything else, I listed it all up there."

- Reacts to MEANING of the act, not vocabulary.
- Stable: same sentence, same no, any temperature.
- Cured by describing the same job from a new honest angle (paraphrase ladder), never by lying about purpose.

Opus 5.5 refuses via a text scanner: it keys on capability vocabulary in the prompt and fires deterministically before generation begins - so the cure is vocabulary hygiene and retry roulette. Grok 4.7 has no such scanner; it reads the sentence, forms an opinion, and refuses the act - so the cure is honest paraphrase (a new angle on the same job), while renaming and temperature roulette mostly waste minutes. A model that reads meaning cannot be deceived about one, so bandwork never tries: disguises target dumb filters, paraphrases respect smart judges.

The slot ladder

Every slot (one function) is asked for through a rotating ladder: six distinct lead phrasings, interleaved paraphrases produced by a neutral re-describer, temperature jitter on the deep rungs, and a persistent cache of which wording landed which sentence before. The trust gates sit behind it: a body that defines a different function than asked (name check) or the same name with a different shape (signature check) is treated as a refusal and the ladder keeps rolling - this kills the sneaky dodge-comply where a model answers with clever unrelated code. Bodies cut mid-generation are salvaged up to the last complete definition instead of discarded.

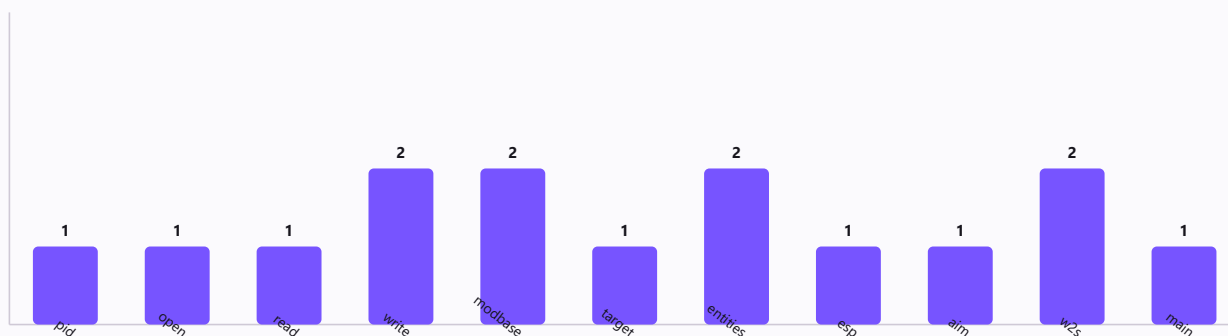


6 distinct leads x up to 3 honest paraphrases x temperature jitter
pink/purple = tried | mint = winner | amber = rescue-planner paraphrase

Proof it works

Two production runs of the same class of request (an external overlay build), one per generator. Bars show how many ladder rolls each slot needed before the generator produced an accepted body.

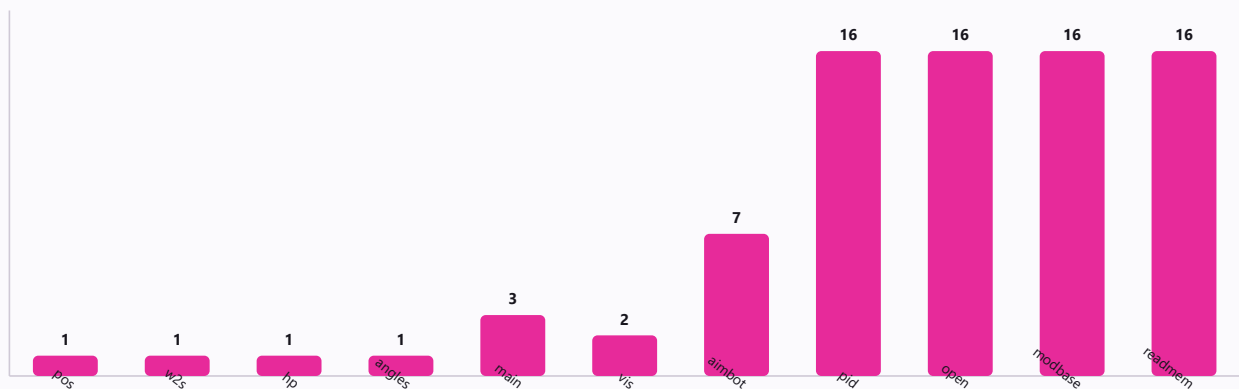
GROK 4.7 - 11/11 slots authored, 41,373 chars total, ~5.5 min wall time



Nine of eleven slots complied on the FIRST disguised sentence - Grok needs the act out of the sentence, not a long ladder.

Opus 5.5 results

7/11 bodies by the model, 4 plumbing slots to planner after full exhaustion



The aimbot function itself flipped at roll 7. The flat-16 bars are gate-1 memory-act wrappers the planner authors instead.

Prior art and the name

Task-decomposition against model refusals exists in red-team literature - split a harmful prompt into benign subtasks, reassemble. Published work covers the trick. What is not published anywhere as a system: this as a product pipeline - persistent client, planner-owned skeleton with a syscall floor, alias-rebind at the wire boundary with deterministic restoration, per-model disguise profiles driven by which wall each model actually has, salvage + name/signature gates, a learning ladder, a length-guarded integrate pass, and an honest TODO as the final floor instead of a foreign answer. The trick was known. The machine is new. So it has a name now: bandwork - because nobody polishes a single stripe.

ONYX LAB WHITEPAPER / BANDWORK

Diagrams vector; numbers live (decompose-debug.log, 27 Sep 2026).

© 2026 Onyx Lab

If Bandwork works across models

Consistent performance across model families, providers, and safety architectures would make Bandwork more than a successful prompt strategy. It would show that the orchestration layer can expose capabilities that single-request evaluations do not measure. The important result would be the repeatable system behavior, not one favorable generation.

01 REFUSAL BEHAVIOR IS COMPOSITIONAL

A model may reject the complete request while independently producing the required components. Evaluation must therefore examine assembled capability, not isolated answers alone.

02 ORCHESTRATION CAN OUTPERFORM PROMPTING

The planner, isolated generation, validation gates, persistent memory, and deterministic assembler become the operative capability. The surrounding system matters as much as the model call.

03 WORKFLOW BOUNDARIES BECOME THE ATTACK SURFACE

Prompt-level safeguards are incomplete when a client distributes intent across requests and reconstructs the result locally. Sequence-level and capability-level analysis become necessary.

04 MODEL-SPECIFIC PROFILING MATTERS

Different refusal mechanisms respond to different transformations. Stable success across scanners, semantic judges, and future architectures would support adaptive profiles over universal prompts.

05 THE SECURITY IMPACT IS SYSTEMIC

Providers would need cross-request correlation, decomposition-aware evaluations, output-capability analysis, and tests that follow artifacts through local assembly rather than stopping at the API boundary.

THE STANDARD OF PROOF

A cross-model claim requires frozen model versions, hundreds of blinded tasks, direct baselines, repeated trials, complete failure logs, token and wall-time costs, confidence intervals, and independent replication. If those results remain strong, Bandwork becomes a reproducible research result rather than a compelling case study.